downstream models. This ensures that the relationships and patterns captured in the embeddings are relevant.

WHY STORE EMBEDDINGS IN A DATABASE?

You may wonder why we should store embeddings in a database and not generate them on the fly for each user query. There are several reasons, but let's discuss just a few:

- *Efficiency*—Generating embeddings can be computationally intensive and time consuming, especially for large datasets. By storing them beforehand, we avoid the overhead of repeated calculations, allowing for faster response times.
- *Scalability*—For applications with a high volume of queries, storing embeddings enables the system to scale effectively. It can handle multiple user requests simultaneously without overwhelming the AI model or the system's resources.
- *Cost effectiveness*—Many AI models, particularly those hosted in the cloud, incur costs based on usage. For example, using the OpenAI embeddings consumes API credits. By storing embeddings once, we can reduce the number of calls made to the AI model, leading to lower operational costs and a more cost-effective solution.

It's worth noting that embedding models are generally deterministic for a specific version of an AI model. This means that if you generate embeddings using the same input and the same embedding model version, you should receive the same output consistently. This behavior is distinct from the generative models used for text completion, which can often produce probabilistic outputs even for the same prompt designed for general purposes. However, if the embedding model version changes or if there are updates to the underlying algorithms or training data, the embeddings produced may differ even for the same input.

5.4.2 Using embeddings in practice: The Vercel AI SDK

The Vercel AI SDK offers two essential helper functions for obtaining embedding values from input text: `embed` and `embedMany`. These functions simplify the process of generating embeddings of a particular AI model using the familiar concepts used in the `generateText` class of functions. The following is a detailed breakdown of each function, including their usage, parameters, and typical use cases.

The purpose of the `embed` function is to generate embeddings for a single input value. This is particularly useful when you need to obtain the semantic representation of a specific piece of text, such as a sentence or phrase. The following code listing shows how to print the embedding vector values of a sentence.

Listing 5.7 Creating a single embedding

```
import { embed } from "ai";
import { createGoogleGenerativeAI } from "@ai-sdk/google";

const google = createGoogleGenerativeAI({ apiKey });

const { embedding } = await embed({
```

Indicates the import of the embed function, which is essential for generating embeddings from text